ADO to GitHub: plan for tonight's call

Source: dev.azure.com/AnalyticsDataHub/Platform. Target: cencora.ghe.com, org ADHPlatform (apps) and ADHPlatform-Modules (Terraform modules). Prepared 11 Nov 2026. Counts are from the 28 Oct 2026 assessment until /ado-snapshot refreshes them.

1. The plan in five lines

- **Timeline:** 12 working weeks, 16 Nov 2026 to 19 Feb 2027, no changes 21 Dec to 1 Jan. Pilot in production by 18 Dec, three waves in Jan and Feb, cutover 15 to 19 Feb.
- **Effort:** about 225 to 495 person-days. The team of 4 at 80% gives about 192. The 12 weeks hold only with 3 extra engineers (vendor) or by moving Wave 3 after February.
- **Pipelines:** convert straight to GitHub Actions with GitHub Actions Importer; no need to convert classic pipelines to ADO YAML first. The 302 classic releases become about 15 shared deploy workflows.
- **Modules and packages:** one repo per Terraform module with semantic version tags (recommended); NuGet and npm to GitHub Packages; Maven and Python stay in Azure Artifacts for now.
- **Blocker:** I only have repo-level rights in ADHPlatform. Org-level runners, repo import, org rulesets and security need the org owner this week.

2. Agenda for the call (30 minutes)

Time	Topic	Outcome needed
5 min	Where we are: kits ready on the VDI, timeline, runner image built	Shared view
10 min	Plan and effort (sections 4 to 9)	Agree the 12-week shape
15 min	Decisions (section 11)	Owner and date per decision

3. Reading ADO with Copilot

Copilot reads ADO through read-only scripts it runs in the VS Code terminal, then writes reports in docs\. Set the PAT first (terminal only, never in chat): $env:ADO_PAT = Read-Host 'Paste ADO PAT'. PAT scopes, all Read: Code, Build, Release, Service Connections, Variable Groups, Secure Files, Environment, Agent Pools, Task Groups, Packaging, Project and Team, Graph, Identity.

What	Prompt in Copilot Chat (Agent mode)	Output
Quick counts for tonight: repos, branches, tags, PRs, builds, releases, service connections, variable groups, environments, feeds	/ado-snapshot	docs\migration\status\<date>-snapshot.md
Full inventory joined per repo	/inventory-all	docs\inventory\<date>-inventory.md
Artifacts feeds, packages, versions, who uses them	/artifacts-inventory	docs\artifacts\inventory\
Effort from real counts	/effort-estimate	docs\migration\effort-estimate.md
My GitHub and ADO permissions + access request email	/check-my-access	docs\security\<date>-my-access.md
Timeline chart	/plan-chart	docs\migration\status\<date>-timeline.md

One-off question without a prompt file:

> Using the CSVs in docs/inventory/raw, list every repo in AnalyticsDataHub/Platform with its branch count, active PRs, build pipelines (YAML or classic), release definitions that deploy it, and service connections those pipelines use. One table, then the 5 repos that look hardest to migrate and why.

4. Repos, branches and PRs

GitHub Enterprise Importer moves each repo with all branches, tags and history. It does not move pipelines, permissions or open PRs.

- Move all branches. Before the wave, delete stale branches in ADO (no commit in 180 days, listed by /ado-snapshot) with the repo owner's OK.
- Active PRs (10 at the assessment): merge or close them before the repo's cutover, or recreate them on GitHub from the same branch (the branch moves with the repo). Completed PRs stay readable in ADO for 90 days.
- Per repo: gh ado2gh migrate-repo --ado-org AnalyticsDataHub --ado-team-project Platform --ado-repo <repo> --github-org ADHPlatform --github-repo <repo> --target-api-url https://api.cencora.ghe.com --target-repo-visibility internal. Needs the migrator role (section 10).
- Check with /repo-verify <repo>: branch count, tag count and last commit must match.

5. Service connections to OIDC

The 103 service connections become a smaller set of Entra identities (about one per app and environment) that trust GitHub through OIDC, so no secrets are stored. Run /oidc-plan after the inventory; it writes every command.

```
az identity create -g rg-github-oidc -n id-gh-<app>-<env>
az role assignment create --assignee <principalId> --role <same role as today> --scope <scope>
az identity federated-credential create -g rg-github-oidc --identity-name id-gh-<app>-<env> -n
gh-<repo>-<env> `
  --issuer https://token.actions.cencora.ghe.com `
  --subject repo:ADHPlatform/<repo>:environment:<env> --audiences api://AzureADTokenExchange
```

Non-Azure connections (SonarQube, Docker Hub, generic) become Key Vault secrets or environment secrets. Connections unused for 90 days are dropped.

6. Build and release pipelines to YAML

There is no need to convert classic pipelines to Azure DevOps YAML first: GitHub Actions Importer reads YAML builds, classic builds and classic releases and writes GitHub Actions YAML directly.

Item (28 Oct count)	How	Prompt / command
125 YAML builds	Actions Importer dry-run, Copilot fixes the gaps, green run on main	/pipeline-convert <id>
17 classic builds	Same, plus a manual check of task settings	/pipeline-convert <id>
302 classic releases	Group into about 15 shared deploy workflows (Terraform deploy, web app, function, SQL …) in ADHPlatform/platform-workflows; each release becomes a short caller workflow with GitHub environments for approvals	/pipeline-patterns, then /pipeline-convert
Task groups, YAML templates	Composite actions and reusable workflows	/pipeline-patterns

57 variable groups, secure files	Environment variables, Key Vault secrets	/library-plan
106 environments with approvals	GitHub environments with required reviewers	/permissions-plan

```
gh actions-importer audit azure-devops --output-dir docs\inventory\importer-audit
gh actions-importer dry-run azure-devops pipeline --pipeline-id <id> --output-dir out\dry-run
gh actions-importer dry-run azure-devops release  --pipeline-id <id> --output-dir out\dry-run
```

Actions Importer runs in Docker. If the VDI has no Docker, run it from the workflow .github\workflows\actions-importer-audit.yml in the kit.

7. Terraform modules: one repo each or one repo for all?

	One repo per module (recommended)	One repo for all modules
Example	ADHPlatform-Modules/terraform-azurerm-keyvault, tag v1.4.2	ADHPlatform-Modules/terraform-modules, tag keyvault/v1.4.2
Consumer source	git::https://cencora.ghe.com/ADHPlatform-Modules/terraform-azurerm-keyvault.git?ref=v1.4.2	git::https://cencora.ghe.com/ADHPlatform-Modules/terraform-modules.git//keyvault?ref=keyvault/v1.4.2
Versions	Independent per module, simple v* tags	Independent only with prefixed tags and release-please manifest mode
Owners and permissions	Per module: CODEOWNERS, team access, rulesets	Shared repo; CODEOWNERS per folder only
Upgrades	Dependabot sees each module's tags	Prefixed tags are harder for Dependabot
CI	Small, runs only for that module	Must detect which module changed
Repos to manage	About 40	1
Downloads in terraform init	Only the module	Whole repo per module reference
Effort to set up	Higher once (template repo makes it fast)	Lower once, higher over time

Decision: Recommendation: one repo per module, as already proposed. Choose one repo for all only if the modules are always released together by one small team.

Steps per module (Copilot: /module-split, /module-tags):

1. Create the template repo terraform-azurerm-template once (kit: templates\module-repo): CI with fmt, validate, tflint, checkov; release-please; CODEOWNERS; PR title check.
2. Split with history: git clone <ado url>; git filter-repo --subdirectory-filter modules/<module>; gh repo create ADHPlatform-Modules/terraform-azurerm-<module> --internal --template ADHPlatform-Modules/terraform-azurerm-template; git push.
3. Backfill tags from versions already published (Universal packages or existing tags), using pipeline provenance to find each version's commit. No invented versions.
4. Version rule: breaking change to inputs or outputs = MAJOR, new feature = MINOR, fix = PATCH. Commits feat:, fix:, feat!: drive release-please, which creates the tag, GitHub Release and CHANGELOG.
5. Protect v* tags with a ruleset (no delete, no move). Consumers pin exact tags; Dependabot opens upgrade PRs.
6. Workflows read modules through a GitHub App token (GITHUB_TOKEN cannot read another org).

8. Azure Artifacts feeds

Package type	Target	Why

NuGet	GitHub Packages: nuget.cencora.ghe.com/ADHPlatform	Supported on GHE.com
npm	GitHub Packages as @adhplatform/<name>	Supported; packages must be renamed to the org scope
Terraform modules as Universal packages	Module repos with tags (section 7)	Git tags are the Terraform-native way
Other Universal packages	Git tags, GitHub Release files, or OCI artifacts in containers.cencora.ghe.com	No Universal Packages in GitHub
Maven / Gradle, Python	Stay in Azure Artifacts for now, read through OIDC	Not available in GitHub Packages on GHE.com
Cached copies of nuget.org / npmjs	Not migrated	Use the public registries

Move only versions still in use: the latest of each major version, anything a repo pins, anything downloaded in 90 days, and anything in the @Release view. Prompts: /artifacts-inventory, /artifacts-plan, /package-migrate <feed> <package>. While ADO pipelines still use a feed, new versions are published to both places.

9. Runners at org level

The runner image (Azure Container Apps, ephemeral, scales to zero, labels self-hosted and aca) is ready and now also works at repo level, so the pilot can start before org rights arrive.

#	Step	Who
1	Create a custom org role with "Manage organization runners and runner groups" and assign it to the migration team (Org settings > Organization roles)	Org owner
2	Create a GitHub App: org permission Self-hosted runners read and write; repo permissions Actions read, Metadata read. Install it on ADHPlatform. Note App ID and installation ID; download the private key	Org owner (or us with role 1 plus app-install rights)
3	Store the private key in Key Vault	Us
4	Deploy: RUNNER_SCOPE=org with APP_ID, INSTALLATION_ID, APP_KEY_FILE, then infra\deploy.sh (az acr build, no Docker needed)	Us
5	Create runner groups aca-nonprod and aca-prod; limit aca-prod to repos that deploy to production	Us with role 1
6	Test: a workflow with runs-on: [self-hosted, aca]; the runner appears, runs one job and is removed	Us
7	Add the tools the pipelines need (/runner-tools reads today's ADO agent capabilities)	Us

10. Checking and requesting my permissions

Run /check-my-access in Copilot (writes the table and a draft email to the org owner), or the commands yourself:

```
gh auth status --hostname cencora.ghe.com
gh api /user/memberships/orgs/ADHPlatform --hostname cencora.ghe.com --jq '{state, role}'
gh api /user/teams --hostname cencora.ghe.com --paginate --jq '.[] |
select(.organization.login=="ADHPlatform") | {team: .name, permission}'
gh api /orgs/ADHPlatform/repos --hostname cencora.ghe.com --paginate --jq '.[] | {name,
admin: .permissions.admin}'
gh api /orgs/ADHPlatform/actions/runners --hostname cencora.ghe.com --jq .total_count   # 403 =
no org runner rights
```

role "admin" = org owner; "member" = not. A 403 on the runners call confirms org runner rights are missing.

Portal checks:

- GitHub: profile picture > Your organizations > ADHPlatform shows Owner or Member. Org Settings visible only to owners. Repo > Settings tab visible = repo admin.
- Azure DevOps: Project settings > Permissions > Users > your name shows your groups (Project Administrators, Contributors ...). Organization settings > Permissions shows Project Collection Administrators.

What to ask the org owner for (we request it; we do not work around it):

Access	Why
Custom org role: Manage organization runners and runner groups	Org-level runners (section 9)
Install the runner GitHub App on ADHPlatform	Runner sign-in without personal tokens
Migrator role: gh ado2gh grant-migrator-role --github-org ADHPlatform --actor <my login> --actor-type USER --target-api-url https://api.cencora.ghe.com	Import repos with history (section 4)
Create repos and teams in ADHPlatform and ADHPlatform-Modules (or create ADHPlatform-Modules)	23 app repos, about 40 module repos
GitHub Secret Protection and Code Security enabled (enterprise owner)	Secret scanning, push protection, CodeQL, Dependabot
Org rulesets, or an owner to apply ours	One branch protection standard
Audit log streaming (enterprise owner)	Keep logs beyond GitHub retention
Azure: User Access Administrator on target subscriptions for setup	OIDC identities and role assignments

11. Effort estimate

Person-days from the 28 Oct counts. /effort-estimate recalculates from live counts. Assumptions: about 35 OIDC identities, 40 modules, 200 module references, about 15 shared deploy workflows, pilot plus 3 waves.

Area	Count	Low (days)	High (days)
Inventory and wave plan	1	4	6
GitHub org setup	1	4	6
Org runners	1	6	10
Repos (import, verify, CODEOWNERS)	23	7	14
Active PRs	10	1	2
Service connections to OIDC	~35 identities	10	21
Variable groups	57	11	23
Environments	106	5	11
YAML builds	125	38	94
Classic builds	17	13	26
Shared deploy workflows	~15	30	60
Classic releases wired to them	302	45	121
Terraform modules	~40	20	40
Module consumer updates	~200	10	20
Artifacts feeds	5	5	15
Security scanning	1	3	5
Cutover and hypercare	4	12	20
Total		**224**	**494**

Reading: Capacity: 4 people x 80% x 60 working days = 192 person-days. Gap: 32 to 302 person-days. Adding 3 vendor engineers for the 12 weeks adds about 180, which covers up to about 370 (192 + 180): the low to middle of the range.

Otherwise move Wave 3 (Modernization_Terraform, 81 builds) after 19 Feb. The biggest lever is the release work: reusing shared workflows well is what keeps it near the low figure.

12. Decisions needed tonight

#	Decision	Recommendation
1	Who is the ADHPlatform org owner, and can they grant section 10 this week?	Name and date tonight
2	Modules: one repo per module or one repo for all	One repo per module
3	Packages: hybrid (GitHub Packages + Azure Artifacts for Maven/Python) or one third-party registry	Hybrid, revisit after the feed inventory
4	Classic releases rebuilt as shared workflows, not one by one	Yes
5	Capacity: vendor engineers, or Wave 3 after February	Decide after /effort-estimate on live counts

13. Tomorrow on the VDI, in order

1. /check-my-access, and send the access email

2. /ado-snapshot, then /effort-estimate

3. /inventory-all, /plan-waves

4. /artifacts-inventory

5. Pilot runner at repo level (runner kit README, "Repo-level runners")